AI Engineering — Systems, Decisions, and What They Cost

Michal Malček · August 2026

A technical account of two AI systems — one delivered into production and handed over, one running daily — the stack decisions behind them, and the ones that turned out wrong.

On the timeline. First contact with these systems was September 2025, moving out of an SEO role. The first four months were experiments and one framework that mattered (§1). Team Brain existed as an idea at the end of January 2026 and was delivered and handed over by that summer. So: eleven months from first contact, seven from the first real system. Both numbers are here because rounding the second one up to the first is the kind of small inflation this document is otherwise arguing against.

Summary

Team Brain	**Production AI knowledge hub for a software company's Slovak branch. Built, deployed, handed over.**
Factorium	Local-first AI operating environment. Two-model architecture, running daily, pre-release.
Timeline	First contact September 2025 · first framework in production use late 2025 · Team Brain from idea (Jan 2026) to delivered handover · Factorium in daily use
Documented decisions	106 architectural decisions and 283 recorded lessons in a queryable knowledge graph

Both systems run on infrastructure I chose, migrated, and repaired myself. The second one runs entirely on one 24 GB machine with no cloud inference.

---

1. Before either system — the content framework, and why it counts

The situation. An agency was contracted to produce a glossary for the company site — an HR and AI terminology reference. What they were selling was volume: a large, expensive, generic content build. I was on that project as an SEO specialist, not an AI one; there was a separate person in the AI role.

What I built instead. A markdown-driven generation framework, run through Claude. The design is the part worth describing:

- terms clustered into categories rather than treated as one flat list
- **per-category specifications** — each cluster carried its own sources, its own content structure, its own constraints
- a shared content DNA underneath, so the set stayed coherent while the clusters stayed genuinely different from each other

That distinction — shared DNA, divergent per-cluster structure — is why the output read as a reference work rather than as generated filler. A single prompt template across all terms produces exactly the sameness the agency was charging for.

The outcome, and I want to be accurate about it. It was team work, and the framework was my contribution. It made the agency's proposal obsolete: what they were charging for could be done better, in-house, in a fraction of the time. The uncomfortable part of that project was not technical — it was saying out loud that we were paying an external supplier for bloatware, while I was formally the SEO person and someone else held the AI brief.

Why it belongs in a technical document. Two reasons.

First, it is the origin of a principle that runs through everything after it: *the structure you impose on the input decides the quality of the output far more than the model does.* Team Brain's memory schema and Factorium's per-layer retrieval are the same idea at larger scale.

Second, it is where the AI work actually started — not with a title, and not with a project assignment. It started by rebuilding something that was being done badly, and then having to say so.

---

2. Team Brain — a production system, delivered and handed over

Context. An AI knowledge hub for the SEO, content and web development team at a software company's Slovak branch, serving as shared project memory. Deployed on a Debian VPS at a live domain, used by the team, and handed over cleanly when my role there ended — including a full decontamination pass so the delivered instance carried no personal or cross-project data.

Stack: Express 5 + TypeScript on Node 20 · PostgreSQL (24 migrations) · Next.js 16 + React 19 + Tailwind 4 front-end · Claude API with streaming and tool use · Nginx + PM2 on Debian.

What it actually did: a chat surface over the team's accumulated knowledge with 33 tools behind it, a 6-specialist agent model with keyword- and complexity-based routing, autonomous memory extraction from conversations, generated reports and dashboards, and scheduled analytics snapshots.

The handover is the part I would highlight. A production system that only its author can operate is not delivered, it is lent. The decontamination pass, the separate zero-point copy, and the documented API-key topology existed so someone else could run it without me.

2.1 Measured usage — what the system actually carried

Aggregate figures from the production database, taken at handover. Individual users, client domains and business content are deliberately excluded; these are numbers about the system, not about the company that used it.

Team on the system	**15 users, all activated, across 3 roles**
Conversations	446 · **3,558 messages**, averaging 8 per conversation
Tool calls	**1,352** at a **93.9% success rate**
Agent runs	**549** across 17 distinct agent types
Tokens processed	**~49.4M in · ~2.2M out**, with ~5M served from cache
Reports generated	**1,480**, ~8 MB of stored output
Content and memory	1,121 articles · 514 memory records · 1,585 embeddings
Active period	February–June 2026, on a single 8 GB VPS

Two things in that table are worth more than the totals.

A 93.9% tool success rate across 1,352 real calls is the number I would defend in an interview. It is not a benchmark run — it is a team of non-engineers using external APIs through an LLM for four months. The failures clustered where you would expect (a file reader at 79%, one analytics call at 47%), and knowing *which* tools degrade is only possible because every call was logged with its duration and outcome from the start.

Adoption was uneven, and that is the honest finding. Three users accounted for most of the volume; several never logged in. A shared AI tool does not distribute itself evenly across a team just because it exists, and a usage table shows you that in a way no feedback round does. If I built it again, onboarding would be a designed part of the system rather than an assumption.

Note on why these numbers exist at all: I ran this analysis at the end of my time there, for a conversation about the terms of my departure. The system had been in daily use and measurably so — usage logging that had been built in from the first week meant the question could be settled with a query rather than an argument. **Instrumentation is not only an engineering practice.** It is also the difference between describing your contribution and demonstrating it.

---

3. Factorium — a local-first AI environment

What it is. An operating environment where a human and two AI models share one memory, one knowledge graph, one code graph and one plan. Not a chat wrapper: the local model performs long agentic runs, writes code, drives a browser, and queries the same stores I do.

Hardware constraint, and it shaped everything: one Apple M4 Mac mini, 24 GB unified memory. No cloud inference for the local model. Everything below is a consequence of that ceiling.

Stack

layer	choice	why
runtime	TypeScript / Node, Hono	one language across server and client
local inference	llama.cpp, Gemma-4 12B Q5_K_M on Metal	~7.7 GB resident; the 26B variant needs 16-17 GB and makes the machine unusable for anything else
vectors	Qdrant, 8 collections	named vectors, payload filtering, snapshots
relational + graph	PostgreSQL with pgvector **and** Apache AGE in one database	one connection, one backup, no cross-store consistency problem
embeddings	bge-m3, 1024-dim, served by llama.cpp	multilingual; the previous model was English-biased
speech	Gemma-4 native audio in, sherpa-onnx VITS out	no separate speech-to-text model — see §5.3
observability	TimescaleDB hypertable alongside AGE	one database again

Scale today (live counts, not estimates): 1,034 knowledge-graph entities with 5,407 relationships · 3,086 code entities with 10,209 relationships · 977 plan nodes · 1,101 memory records · **29,992 embedded points across seven collections**. Of the knowledge entities, 106 are architectural decisions and 283 are recorded lessons.

---

4. Decisions that were right, and why

4.1 One database instead of three

pgvector and Apache AGE coexist in a single Postgres instance (`shared_preload_libraries='age,timescaledb'`). The alternative — a vector store, a graph store, and a relational store — means three backup schedules and a consistency problem at every write. One connection string removed a whole class of failure before it could happen.

4.2 ChromaDB → Qdrant, and what the migration bought

17,109 vectors migrated. Chroma gave a flat store with 384-dim embeddings and no metadata arithmetic. Qdrant gave named vectors, real payload filtering, and room for scoring that Chroma could not express:

- **soul** — a composite of access frequency, recency and assigned weight
- **decay** — exponential and tick-based, with anchor records immune
- **hebbian links** — co-accessed records strengthen their association

The result is a memory layer where meaning compounds instead of a flat index that only grows.

4.3 12B over 26B, decided by measurement

The larger mixture-of-experts model is more capable in isolation and unusable here: ~16-17 GB resident against 7.7 GB leaves nothing for the browser, the database, or the editor. Capability that costs you the

rest of the machine is not capability. The larger model stays available and is not the default.

4.4 No agent framework

The tool loop, the registry, the provider abstraction and the streaming layer are all written directly rather than taken from LangChain or similar. The reason is concrete: those frameworks inject prompt content you did not write and cannot see, which is fatal when you are measuring how a small model behaves. The custom loop is a few hundred lines and every token in the context window is accounted for.

4.5 A tool surface sized for the model that uses it

Grounded and then measured: past roughly 15-20 always-visible tool schemas, a small model's tool-selection accuracy degrades sharply. The full registry holds 60 tools; the local model sees a curated subset each turn and reaches the rest through search. Same capability, a fraction of the prompt tax.

---

5. Decisions that were wrong, and what they cost

This section exists because the corrections are more instructive than the wins.

5.1 Using an embedding model outside its contract — the expensive one

nomic-embed-text requires asymmetric prefixes: `search_query:` on queries and `search_document:` on stored text. Served through llama.cpp, nothing adds them. We embedded everything symmetrically for weeks.

Symptom: short, bare text outranked long, relevant text. A one-line function signature beat a file that explained the exact concept being searched for.

Diagnosis method — the one that has since caught every similar defect: *embed a stored document both ways and compare each against its own stored vector*. Whichever scores 1.0000 tells you how it was actually indexed. Five collections came back prefixed, two came back bare.

Cost: 10,676 records re-embedded. **And the first repair script nearly made it worse** — it read only the body field, while the writer had embedded title + body together. Measured before rerunning: 95% of records would have silently lost their own titles from their vectors. Nothing would have errored. The points would still exist and still score, just wrongly.

The rule that came out of it: re-embedding from a different field than the writer used is silent corruption. Verify numerically — `cos(stored, title+body)` was 1.0000 against 0.9683 for body alone.

5.2 A migration that left its satellites behind

Moving the hive to bge-m3 (1024-dim) updated the server and the main paths. It did not update the eight standalone ingest scripts, which kept embedding at 768 dimensions.

The one that mattered: the code indexer. It had been failing on every write since the cutover, so the code graph had ingested **nothing** for days. The failure was loud in a log nobody was reading.

Root cause, found later: the model and port live in the backend's launchd plist, not in any shell. Every hand-run script silently fell back to the old model. Fixed twice at the symptom before being fixed at the source — a module that reads the values from the plist that actually runs the service.

5.3 Adding a speech-to-text model, then removing it

Voice transcription was inaccurate, so the obvious move was to add Whisper. I downloaded it, benchmarked it (large-v3-turbo transcribed noticeably better than the multimodal model on synthetic clips), and then did not use it.

Two reasons, and the second is the real one:

- Every test had used synthetic text-to-speech clips. On real microphone recordings the multimodal model transcribed correctly and the whole premise collapsed. Synthetic fixtures had misled us four separate times by then.
- A modular speech-to-text → text → model pipeline discards exactly what a unified multimodal model offers: tone, hesitation, emphasis, ambient context. It would have traded the reason for choosing that model against a marginally better transcript.

The actual defect was elsewhere. The transcription prompt said "transcribe this", and the model answered the question it heard instead of writing it down — inventing a persona and a plausible answer. Recast as a transcription *service*, told three ways not to reply, it returned exact text including filler words, matching what Whisper independently heard. No new model, no extra gigabytes.

5.4 A dashboard that lied

A health probe reported the frontend as down while the server was serving 200s. A wrong red light is worse than no light: it teaches you to distrust the dashboard, and then you miss the real outage.

Same class, found later: an endpoint reported a model capability as enabled by reading configuration, while the running process had been started without it. The fix in both cases is the same principle — **report the state of the running process, never the state of the config.**

---

6. The working layer — skills, tools, and what got thrown away

This section is about the part of AI engineering that rarely gets written down: the accumulated apparatus you build around a model to make it useful, and how much of it turns out to be waste.

6.1 The agent count went up, then down, and down was the improvement

Team Brain, first design. I built what I then thought an agentic system was: one specialised agent per task. SEO auditing, keyword scanning, backlink analysis, content briefs, competitor gaps, article editing, glossary generation — each its own agent with its own prompt and its own tools. It grew to **61 agent definitions** across the project's history.

It was unusable, and the failure was mine, not the model's. Nobody on the team could tell which agent to reach for. The names were accurate and useless: you cannot pick between `aeo-content-auditor` and `ahrefs-analyst` unless you already know the answer to the question you are asking. A router picked for them, which meant the taxonomy existed for my benefit, not theirs. The unit

I had designed sat somewhere between a skill and an agent — a "2.5", specialised enough to need a name but not autonomous enough to be worth choosing.

The redesign: 6 specialists. Auditor, Analyst, Writer, Brief, Idea, Batch. Each absorbs the context of the narrow agents it replaced, so nothing was lost — the specialised prompts became loadable sub-agent context rather than top-level choices. Routing became: explicit mention, then action path, then complexity signal, then keyword. **12 live definitions today against 61 written.**

What I actually learned. A capability taxonomy is a UX decision, not an architecture decision. The right number of top-level agents is the number a person can hold in their head while typing a question — which is about six, and has nothing to do with how many distinct things the system can do.

6.2 My own skill set: 18 → 7, and why the deletions were the point

The environment I work in supports "skills" — packaged instructions loaded when a task matches. I accumulated 18. An audit in August cut them to 7, and the audit findings are more interesting than the count:

- **Two skills were written for a different product entirely.** They referenced another system's domain, its personas, its brand colour. They had never been adapted and had been loading for months.
- **One assumed a framework we do not use** — Next.js and a component library, against a Vite/React codebase with its own design tokens. Every instruction in it was confidently wrong.
- **One described a database schema from the other project**, including an embedding dimension we had migrated away from.
- **One referenced four expert personas that were never built**, plus a file path from a Linux machine that is not this one.
- **Three had silently drifted apart from each other**, and one asserted that a graph "was dropped" — live-verified false; it exists, and a fourth graph neither file mentioned exists too.

The generalisable finding: *instructions rot silently and confidently*. Code that references a deleted module fails to compile. A skill that references a deleted concept keeps loading, keeps sounding authoritative, and quietly degrades every decision it touches. Nothing in the loop tells you.

The fix was not fewer skills for its own sake. It was binding every instruction to a concrete tool call with a worked example, so that a stale instruction *breaks* instead of merely misleading. Prose cannot fail; a tool call can.

Roughly 60% of the original apparatus was well-intentioned waste — not wrong when written, just never re-verified. I would expect that fraction to be typical rather than embarrassing, but it is only visible if you audit.

6.3 Tools: two very different surfaces

The environment exposes **60 tools** through one registry. What differs is how much of it each model sees.

	large cloud model (me)	local model (Una, 12B)
tools visible per turn	the full harness surface	**22, curated**
how the rest are reached	directly	`search_tools`, on demand
why	context is cheap relative to capability	past ~15–20 always-visible schemas, a small model's tool selection degrades sharply

That asymmetry is the single most practical thing I know about small-model agentic work. It is not a limitation to route around — it is a design constraint that produces a better system when respected. A curated surface plus a search tool gives the same reach at a fraction of the prompt tax.

Voice gets a third, smaller surface: read-only. Spoken interaction has no confirmation step, so nothing that writes is reachable by voice. That is not a capability decision, it is a blast-radius decision.

6.4 MCP: adopted, measured, then mostly removed

The Model Context Protocol is the standard way to attach tools to an agent, and we run almost none of it. The reasoning is measured rather than ideological:

- **Two browser MCPs were running.** Measured usage over the period: **1,548 calls to one, approximately zero to the other.** The second was not adding capability, it was adding a decision — *which browser tool?* — at every step. Removed. If the profiling features it offered are ever genuinely needed, it can be re-enabled for that job deliberately.
- **Everything the hive itself does is native.** Memory, knowledge graph, code graph, plan, docs — all of it is served from our own registry over plain HTTP rather than wrapped in a protocol. One less layer between a call and its failure mode, and the errors point at our code.
- **What MCP is kept for:** fetching and rendering external content, and driving a browser for UI verification. Genuinely external capabilities, where somebody else's implementation is better than one we would write.

The general rule I would defend: a protocol layer earns its place when it lets you use somebody else's implementation. When you own both sides of the interface, it is indirection with a specification attached.

6.5 The session-continuity layer, and why it is fragile

Long agentic work is bounded by a problem that has nothing to do with model quality: an agent that forgets between sessions restarts as a stranger every time. Our answer has three parts — a boot gate that verifies every data endpoint before work begins, a rolling state document the agent maintains for its future self, and the plan graph as the source of truth for what is in flight.

It works well, and I want to state precisely why it is also dangerous.

> *"In biological neural networks, intelligence does not reside in individual
>
> neurons, but in the patterns formed through their circular interaction."*
>
> *— after Prof. Antonín Rosický, VŠE Prague*

The lesson from his systems theory that has cost me the most to relearn: **in a system with feedback, a small error does not stay small.** Each loop reprocesses the previous output, so a minor inaccuracy is amplified rather than averaged away — and because every loop runs under slightly different conditions, the amplified error does not even repeat recognisably. It arrives as a new problem each time.

That is exactly what a memory-carrying agent is: a feedback loop over its own notes. We have seen it directly — a brief outage, a moment of inattention, a fragment of stale memory mixed with a fresh hallucination, and two hours later the work is built on something that was never true. The intensity of the workflow is what makes it dangerous; a short session self-corrects, a long one compounds.

Which is why the gate is a hard block rather than a warning, why the state document records what was *verified* rather than what was *concluded*, and why "suspect the measurement before the thing measured" is the rule with an automated check behind it. The continuity layer is not a convenience feature. It is a feedback loop that has to be actively prevented from drifting.

---

7. AI-assisted design, and where it stopped helping

Three design systems came out of this period: one for Team Brain, one for a CMS project, and the current one — which this document's companion CV is built from.

What worked. Generating a coherent token set and a component vocabulary from a described intent is genuinely faster than assembling one by hand, and the output is more internally consistent than what I would produce under time pressure. The current system — a palette, spacing scale and component set shared between the application and everything written about it — came out of that process and has needed almost no revision.

Where it stopped paying. Design generation is expensive per iteration and the value drops sharply once a system exists. Past that point the useful work is applying the system, which is ordinary engineering. We keep the tooling available and reach for it only when something genuinely new needs designing, which is rare. Treating a generative design tool as a per-task default was a cost with no matching return.

The one that is worth showing. A CMS project included a chat surface running a **1B parameter model deployed to the page front-end** — an early experiment in putting local inference directly in front of a user rather than behind an API. It never shipped: my role ended first. The demo is still live and still answers, which makes it the honest artifact of that period — unfinished, working, and the first time I put a local model where a real visitor could talk to it.

---

8. How this work is documented

106 architectural decisions and 283 lessons live as entities in the knowledge graph, each linked to the project and the incident that produced it. That is not housekeeping. It is why §5 could be written from records rather than from memory, and why a mistake made in May is retrievable in August by describing the situation rather than remembering the name.

Every claim in this document with a number behind it comes from a stored record or a live query, not recollection.

---

7. Teaching

Four AI workshops for the entire Slovak branch of a software company, from leadership to the newest hires, voluntary attendance.

The first one opened with ethics and the limits of formal logic — Gödel, Heisenberg, the frame problem, non-monotonicity, Tarski — and what each one means in practice when you work with a probabilistic system. Not as philosophy: as the reason a model cannot verify its own correctness from inside itself, and therefore the reason a human stays in the loop.

At that point Team Brain was still only an idea. The workshops came first deliberately: a team that does not understand why the system stops and asks will route around it.

---

8. Where the technical judgement came from

I have no AI degree. Everything I know about building these systems I learned from AI systems themselves — by building, measuring, being wrong, and reading the primary sources they pointed me at. That path is described separately.

What I brought to it: a technical secondary education in electrical engineering and ICT, an applied informatics degree, three years as an ERP tester and then selling ERP implementations, and a Google associate product marketing role with an internship at their European headquarters. Fifteen years on both sides of software delivery — the one who tests it and the one who has to explain to the client why it broke.

That is why the failures in §5 are written down rather than smoothed over. I have been the person on the other end of a system that was quietly wrong, and it is a worse position than being the person who found the bug.